Archivo

s

UNRN

Universidad Nacional
de **Río Negro**

12 / 2026

r33

{ O cómo guardar información }

Apertura

```
FILE* fopen(char* nombre_archivo, char* modo);
```

Si falla, apuntará a `NULL`

modos

UNRN Universidad Nacional de **Río Negro**

"r"

para leer
el archivo debe existir

"w"

para escribir
si el archivo existe, **se reemplaza por uno vacío**

"a"

Lo que escribamos va al final
Si el archivo no existe es creado

"r+"

Para leer y escribir
El archivo tiene que existir

"w+"

crea un archivo vacío para leer y escribir
reemplazandolo si existia por uno vacío

"a+"

Para leer y escribir al final
si no existe, se crea

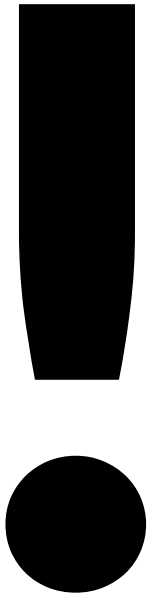
"b"

modo binario, y va junto a si es RBA+.
no le daremos uso

Y todo lo que se abre...



Se tiene que cerrar



Cierre




```
int fclose(FILE *stream);
```

Una forma de verificar que el archivo esta OK

```
FILE *archivo;

archivo = fopen("salida_fputc.txt", "r");

if (archivo == NULL) {
    printf("Error al abrir el archivo\n");
} else {
    // le damos uso
    fclose(archivo);
}
```

Vamos a ver una forma más completa

EOF

Fin del archivo (*End of File*)
Se usa para indicar que nos quedamos sin archivo para leer

Escritura a archivo

UNRN Universidad Nacional de Río Negro

salida a archivo

caracteres impresos ✅ o
valores negativos ❌



```
int fprintf(FILE* stream, ...);
```


*tambien pueden mandar carácter a carácter, pero es mucho más práctico usar esto

Ejemplo mínimo

```c
#include <stdio.h>

FILE *archivo;
archivo = fopen("salida.txt", "w"); // debieramos chequear
fprintf(archivo, "Hola, mundo!\n");
fclose(archivo);
```

De a carácter individual

```
int fputc( int ch, FILE* stream );
```

Devuelve `ch` o `EOF` segun termine

Ejemplo mínimo

```
FILE *archivo;
char texto[] = "Escribiendo caracter por caracter.";
int i = 0;
archivo = fopen("salida_fputc.txt", "w");
while (texto[i] != '\0') {
    fputc(texto[i], archivo);
    i++;
}
fputc('\n', archivo);
fclose(archivo);
```

Lectura de archivo

UNRN Universidad Nacional de Río Negro

Carácter a carácter

UNRN Universidad Nacional de Río Negro

Para leer de a carácter de archivo

un carácter individual ✅

o

EOF ❌



```
int fgetc(FILE *stream);
```

Un ejemplo mínimo

```
FILE* archivo = fopen("salida_fputc.txt", "r");
int caracter;

caracter = fgetc(archivo);
while (!feof(archivo)) {
    putchar(caracter);
    caracter = fgetc(archivo);
}

fclose(archivo);
```

¿El archivo está en el final?

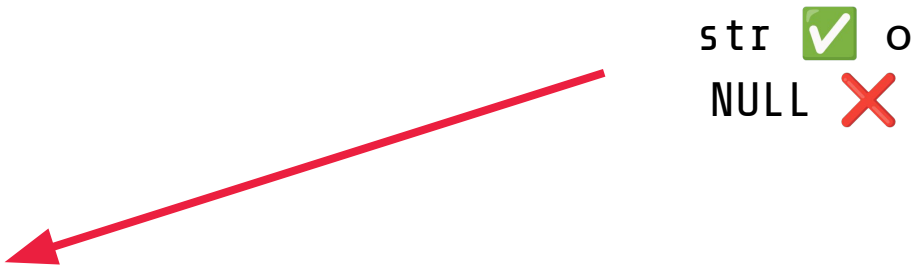
```
1 dale que va
o
0 no hay más archivo
```

```
int feof(FILE *stream)
```

Leer de a bloques

UNRN Universidad Nacional de Río Negro

Para leer de a bloque desde archivo




```
char* fgets(char* str, int n, FILE *stream);
```

De a 10 caracteres

```
#define MAX_CADENA 10

char bloque[MAX_CADENA];
FILE* archivo = fopen("salida_fputc.txt", "r");

while (fgets(bloque, sizeof(bloque), archivo) != NULL)
{
    printf("%s\n", bloque);
}

fclose(archivo)
```

¿Si hay más de MAX_CADENA?

A repetir la lectura hasta `feof`

{ Para archivos de tamaño desconocido es mucho más simple carácter a carácter }

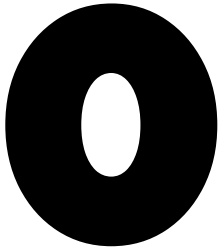

¿Preguntas?

Gestión de errores

UNRN Universidad Nacional de Río Negro

¿Qué puede fallar?

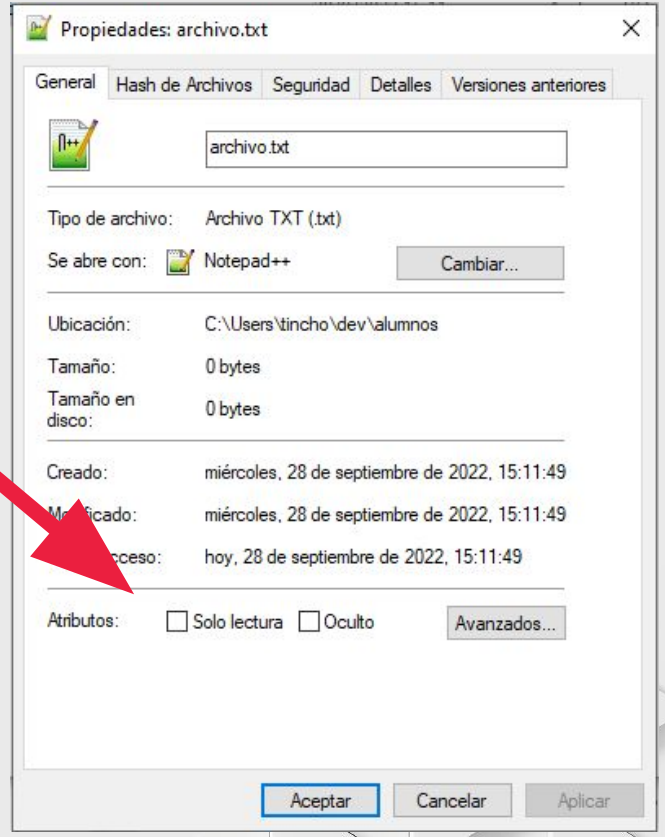
UNRN Universidad Nacional de Río Negro



El archivo no existe

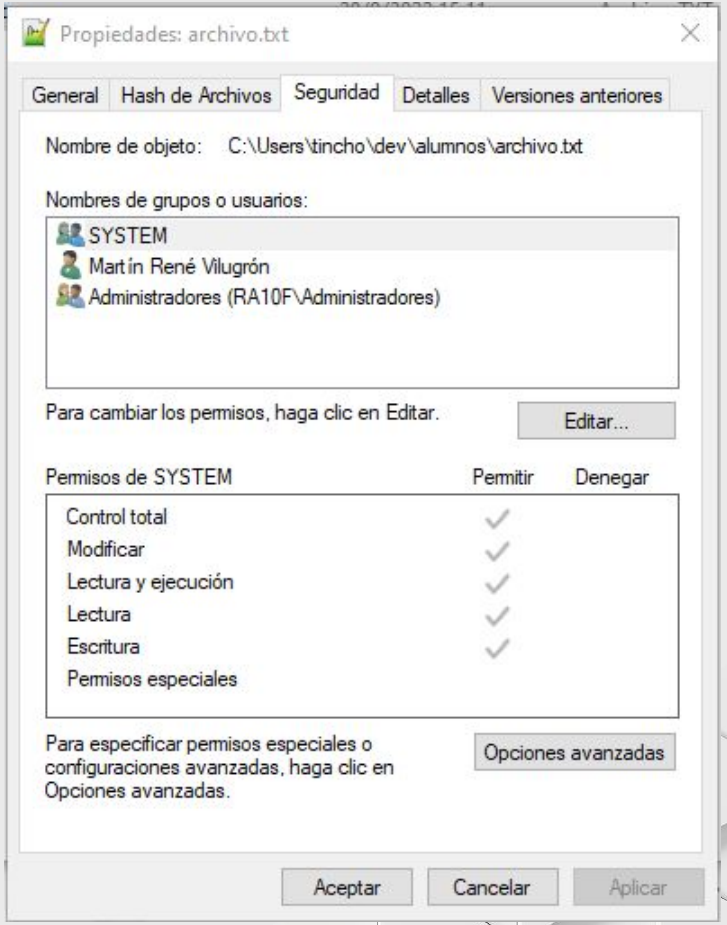
1 Permisos de archivo

El archivo a modificar estaba como ‘solo lectura’



(botón derecho, propiedades)

El usuario no puede acceder* archivos en la ubicación



*leer o escribir

2 Bloqueado por otro programa

El archivo ya estaba abierto*

```
a1 = fopen("archivo.txt", "r");
a2 = fopen("archivo.txt", "w");
```

*en especial si olvidan el `fclose` y necesitan del archivo

Si falla algo, ¿no podemos saber más?

perror en stdio.h

```
void perror(char* mensaje);
```

Imprime por consola (`stderr`) el mensaje sobre el último error junto a un mensaje propio.

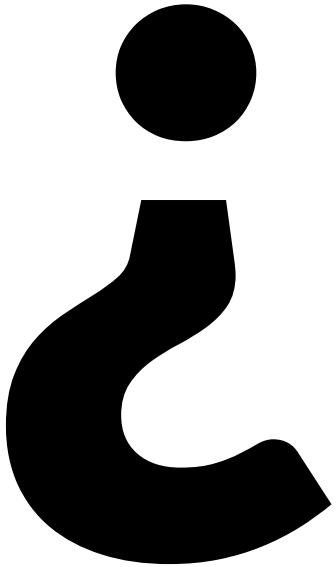
Archivo inexistente

```c
#include <stdio.h>

int main()
{
    FILE *archivo = fopen("no_existi.txt", "r");
    if (archivo == NULL)
    {
        perror("fopen() fallo");
    }
    else
    {
        fclose(archivo);
    }
}
```

Y veremos por consola

```
fopen() fallo: No such file or directory
```



Podemos decidir cómo seguir



errno en errno.h

Una *variable global* con el código de error producido por la última llamada a función.

https://man7.org/linux/man-pages/man3/errno.3.html

1

funciones como `printf` *puede* cambiar su valor

{ Copien el valor de `errno` antes de llamar a otra función }

2

el valor *solo* tiene sentido si la función retorna valores como `NULL` o `-1`

Por lo que se utiliza

```
FILE *archivo = fopen("no_existi.txt", "r");
if (archivo == NULL)
{
    printf("%s\n\n",strerror(errno));
    perror("fopen() fallo");
}
else
{
    // todo OK
}
```



Algunos valores relevantes de `errno.h`

`ENOENT` - El archivo no existe

`EEXIST` - El archivo ya existe

`ENAMETOOLONG` - Nombre muy largo

`EACCES` - Error de permisos de acceso

https://man7.org/linux/man-pages/man3/errno.3.html (son como ~133)



Todos comienzan con E

Y nos dan lugar a decidir cómo seguir

Por ejemplo

Si el archivo ya existía, pueden agregar al namefile un “-copia”


¿Preguntas?

Pero, si necesitamos el mensaje

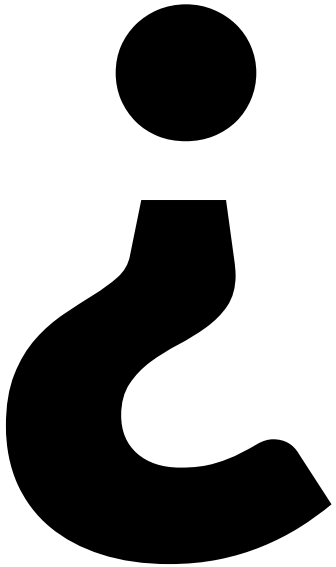
También lo podemos obtener igual que el de `perror`

strerror en string.h

```
char* strerror( int errnum );
```

Retorna un puntero a la cadena que describe el error indicado por `errnum`.

Idéntico al que saldría por `perror`.



Que retorna `fprintf`



fprintf (y printf)

La cantidad de caracteres escritos

Con valores negativos si hay problemas

{ Ojo con usar un FILE cerrado o NULL }

{ **Y aunque todas las operaciones de archivos *pueden fallar*** }

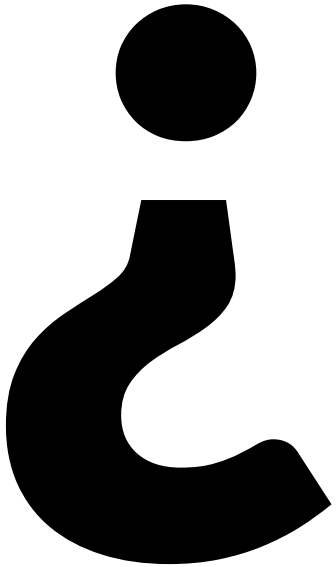
atajarlos en `fopen` es lo más importante

Otras situaciones de error como para ver

- Abrir o cerrar dos veces un archivo
- Operar con un `FILE` en `NULL`
- No cerrar el archivo luego enviar información
- El archivo deja de estar disponible


¿Preguntas?

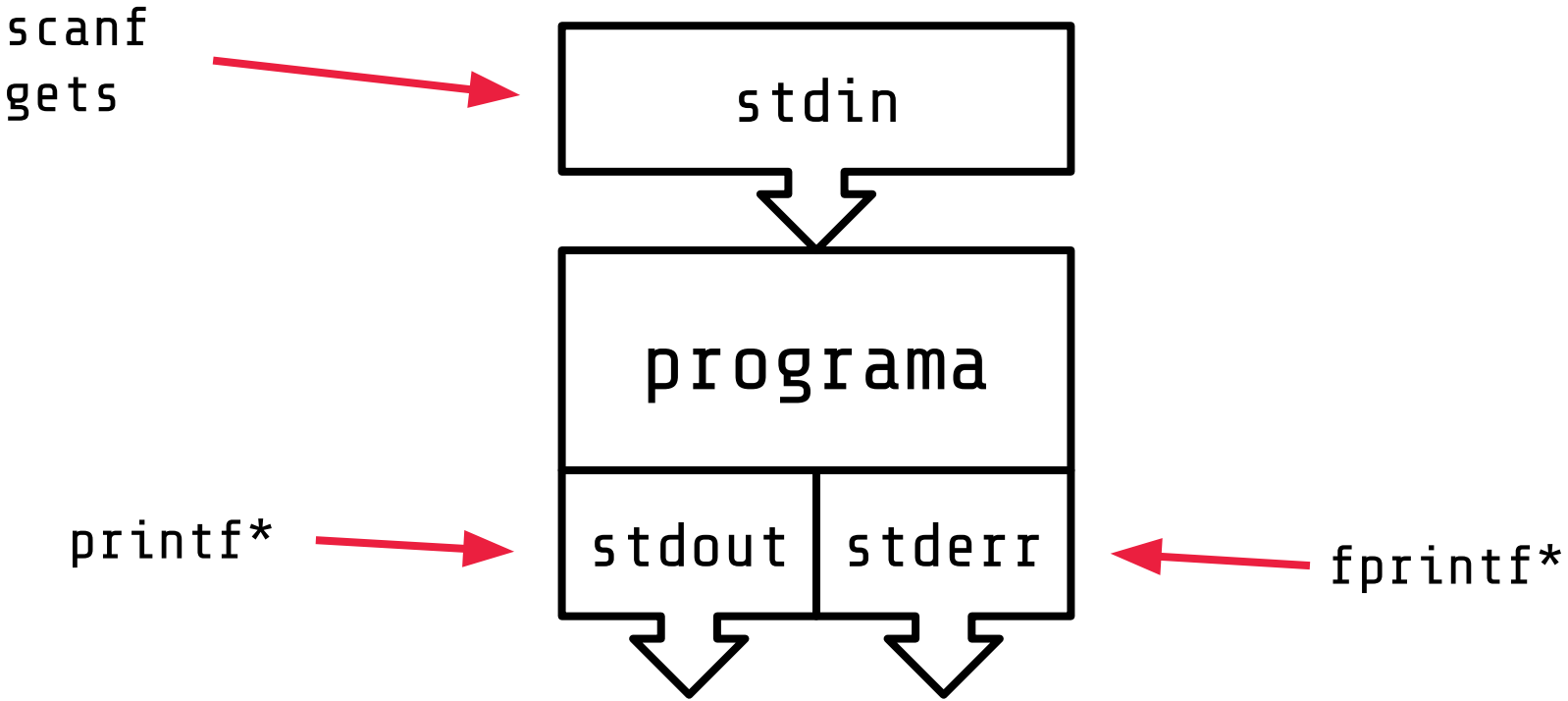

PLUS
ULTRA



Qué es eso de `stderr`



Otro canal de salida como el usado por printf (stdout)


scanf
gets
stdin
programa
printf*
stdout
stderr
fprintf*

Lo normal es que ambos esten conectados a la consola

Y permite separar los mensajes de error de la salida normal del programa

Se usa para los programas conectados por la terminal

Para escribir a ‘error estándar’

```
fprintf(stderr, "Esto es un error!\n");
```

Aunque este ‘superpuesto’ con los `printf` tradicionales

```
$>./miprograma 2> errores.txt
```

Envía *solo* los errores al archivo errores.txt

{ **Hay más combinaciones y posibilidades, pero escapan a este curso** }

Le darán uso en la materia Sistemas Operativos

Lo mismo que printf y scanf

```
fprintf(stdout, "Esto es un printf!\n");
fscanf(stdin, "%d",&numero);
```

Lo interesante es que los tres

son archivos

{ y aplican todos los códigos de error }

Interceptando STDIN/STDOUT es posible interceptar printf/scanf y armar tests

Macros interesantes

UNRN Universidad Nacional de Río Negro

Mensajes de error *extra* detallados

```
fprintf (stderr, "Error interno: "
                 "tamanio de cadena negativa "
                 "%d en %s, linea %d.",
         largo, __FILE__, __LINE__);
```

Para más información al explotar


¿Preguntas?

Hasta la próxima

UNRN Universidad Nacional de **Río Negro**